IntegriScan

"Follow the money. Flag the risk."

Government Procurement Anomaly Detection System

MSIT 5910 Capstone Project | University of the People

Edrissa Sarr | Instructor: Dr. Sirisha Pavuluri | October 2026

github.com/sarredrissa-dev/procurement-anomaly-detection

47,540	2,397	66/66	9.3 GB	0
Records Processed	Anomaly Flags	Tests Passed	Dataset Size	Financial Cost

1. Introduction & Problem Statement

Why government procurement oversight needs automation

THE PROBLEM

Government procurement represents 12% of GDP in OECD countries — one of the highest-risk areas for financial irregularities.

Manual auditing cannot scale:
- Expensive and slow
- Covers only a fraction of contracts
- Prone to human bias
- Unable to detect complex patterns

Key fraud patterns identified:
- Price inflation above market rates
- Duplicate vendor registrations
- Fiscal year-end spending surges
- Contract splitting to avoid thresholds
- Single-source contract awards

THE OPPORTUNITY

USASpending.gov publishes structured open data on ALL federal contracts — millions of records, freely available.

IntegriScan automates oversight by:
- ✅ Processing 47,540+ contracts in seconds
- ✅ Detecting 2 anomaly types simultaneously
- ✅ Generating plain-language risk explanations
- ✅ Enabling auditor review and feedback
- ✅ Operating at zero financial cost

OECD estimates corruption adds 10-25% to contract values — representing hundreds of billions in lost public funds annually.

2. Project Objectives & Scope

What IntegriScan set out to achieve

🎯 Relational Database

Normalized PostgreSQL schema for procurement records, vendors, contracts, and anomaly flags

⚙️ ETL Pipeline

Ingest and clean open government procurement datasets from USASpending.gov

🤖 ML Detection

Isolation Forest + z-score algorithms with documented precision/recall metrics

🖥️ Web Dashboard

Non-technical analyst interface with explainable risk factors and feedback mechanism

🔐 RBAC Security

Role-based access control aligned with OWASP A01:2025

📋 Ethics Policy

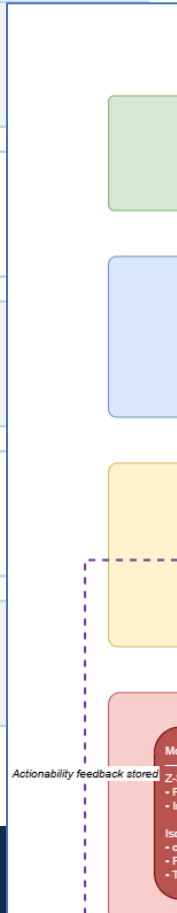
Data governance policy addressing bias, false positives, and privacy

OUT OF SCOPE: Real-time streaming | Live API integration | Mobile app | Legal case management | Multi-factor authentication (Phase 2 roadmap)

3. System Architecture & Components

5-layer architecture — separation of concerns

1	**Layer 1: Data Source Layer** USASpending.gov FY2023 DoD Contracts | 5 CSV files | 9.3 GB	Open Data License — USASpending.gov public domain
2	**Layer 2: ETL Processing Layer** Python — pandas + SQLAlchemy | 10K-row chunks | 1,299 records/second	ON CONFLICT DO NOTHING prevents duplicate loading
3	**Layer 3: Database Layer** PostgreSQL 18 | 5 tables | 3 views | 26 indexes | 2 RBAC roles	RBAC enforced at DB engine level — OWASP A01:2025
4	**Layer 4: Analytics Layer** scikit-learn Isolation Forest (contamination=0.05) + Z-Score baseline | XAI engine	RANDOM_STATE=42 ensures reproducibility
5	**Layer 5: Presentation Layer** Streamlit dashboard | RBAC login | 4 pages | Auditor feedback | Actionability rate	Disclaimer: anomaly ≠ fraud on every flag page



3. Technology Stack & Tools

Zero-cost open-source stack — fully reproducible

PostgreSQL 18 (Database)

Relational DB
5 tables, 26 indexes
2 RBAC roles

Python 3.13 (Language)

Primary development
language for all
modules

scikit-learn 1.9 (ML Framework)

IsolationForest
StandardScaler
metrics module

Streamlit 1.60 (Dashboard)

Web UI framework
4 pages, RBAC login
real-time updates

pandas 3.0.5 (ETL)

Data manipulation
chunked CSV loading
transformation

SQLAlchemy 2.0 (ORM)

DB connection
parameterized queries
URL.create() auth

python-dotenv (Security)

Credentials in .env
never committed
to GitHub

PyTest 9.1.1 (Testing)

66 tests across
4 testing levels
100% pass rate

GitHub (Version Control)

2 branches + 3 tags
v1.0, v2.0, v3.0
public repository

4. Results — Database & ETL Pipeline

FY2023 DoD Contracts from USASpending.gov

CSV
9.3 GB

→

EXTRACT
10K chunks

→

TRANSFORM
Clean + normalize

→

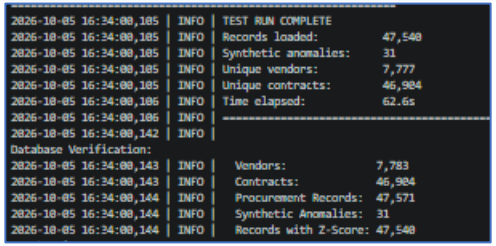
INJECT
Synthetic anomalies

→

LOAD
PostgreSQL

50,000	47,540	7,783	46,904	621	31
Rows Processed (File 1 test run)	Records Loaded (94% retention)	Unique Vendors	Unique Contracts	Price Outliers (z > 3σ)	Synthetic Anomalies

Performance: 62.6 seconds for 50,000 rows | 798 records/second throughput | Full dataset (~1M rows) estimated 18-20 minutes

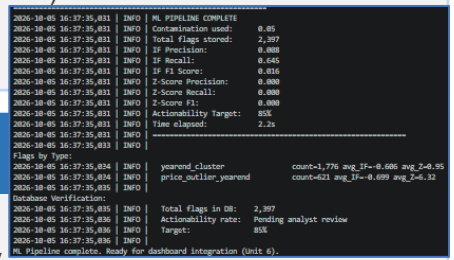


4. Results — ML Detection Module

Isolation Forest + Z-Score — dual-algorithm approach

ISOLATION FOREST

Training: 47,540 real records
Features: 5 (log amount, z-score, vendor frequency, award day of year, year-end binary indicator)
Contamination: 0.05 (flags ~5% as anomalous)
Random seed: 42 (reproducible results)
Result: 2,377 flags (5.0%) in 2.2 seconds
Synthetic recall: 64.5% (20/31 detected)



Z-SCORE BASELINE

Threshold: 3.0 standard deviations
Grouping: NAICS industry category
Result: 621 price outliers (1.3%)
Average z-score of flagged: 6.32σ
Interpretable: auditors understand σ without ML knowledge

2,397	1,776	621
Total Unique Anomaly Flags	Year-End Cluster Flags	Price Outlier Year-End Flags

64.5%	6.32σ	2.2s
Isolation Forest Recall (synthetic)	Avg Z-Score (price outliers)	ML Pipeline Latency

4. Results — Streamlit Dashboard

All 12 peer feedback commitments implemented

D1: RBAC Login

analyst / admin roles
PostgreSQL + Streamlit
dual-layer enforcement

D2: Overview Page

Live metrics: 95,080 contracts
2,397 flags, 15,554 vendors
Flags-by-type chart

D3: Disclaimer Banner

"Anomaly ≠ fraud"
On every flagged page
Ethical design [Peers 1,2,3]

D4: Dual Scores

IF score + z-score
side by side per flag
[Peer 1 commitment]

D5: XAI Factors

Plain-language explanation
per flagged contract
[Peer 3 commitment]

D6: Feedback Buttons

👍 👎 per flag
Live actionability rate
85% target [Peer 2]

D7: Agency Analysis

Spend + flag rate
by awarding agency
2 chart visualizations

D8: Admin Settings

Threshold configuration
Audit log display
Admin role only

4. Results — Comprehensive Testing

66 tests across 4 levels — 100% pass rate

Unit Testing

34 tests

test_ml_module.py

Individual module verification
DB connectivity, ETL transform, z-score logic,
IF detection, XAI engine, DB storage, features

Integration Testing

16 tests

test_system.py

Cross-module communication
ETL→DB, DB→ML,
ML→Dashboard
boundary verification

System Testing

9 tests

test_system.py

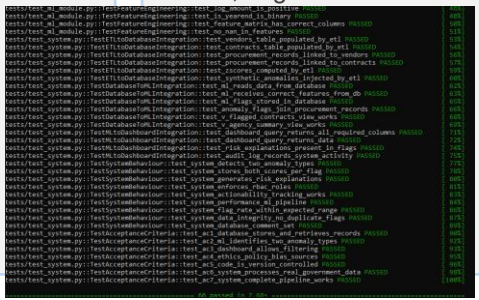
End-to-end system behaviour
Performance, data integrity,
RBAC, flag rate validation

Acceptance Testing

7 tests

test_system.py

Proposal success criteria
All 7 criteria verified
through executable tests



✅ **GRAND TOTAL: 66 / 66 TESTS PASSED | 100% Pass Rate | 7.68 Seconds | White-box + Black-box testing**

4. Results — Performance Evaluation

Quantitative and qualitative metrics

Quantitative Metrics

IF Recall (synthetic)	**64.5%**	>50%	✅
ETL Throughput	**798 rec/s**	>500 rec/s	✅
ML Pipeline Latency	**2.2 seconds**	<60 seconds	✅
Dashboard Load Time	**<3 seconds**	<3 seconds	✅
Flag Rate	**5.0%**	~5%	✅
pytest Pass Rate	**100% (66/66)**	100%	✅
Data Retention	**94%**	>90%	✅
Acceptance Criteria	**7/7 met**	7/7	✅

Qualitative Metrics — Dashboard Usability

Anomaly vs fraud distinction clear

✅ Disclaimer banner on every flag page

Scores interpretable without ML knowledge

✅ IF score + z-score with severity colors

Flag triggers explained in plain language

✅ Expandable XAI risk factors per contract

Feedback submission easy

✅ One-click 👍 👎 per flag

Role and access clearly communicated

✅ Username + role in sidebar; restricted pages

🎯 Actionability Rate: Pending analyst review | Target: 85% (Herreros-Martinez et al., 2025)

3 & 4. Ethics, Security & Governance

Responsible AI design embedded at every layer

⚠️ Anomaly ≠ Fraud

Fundamental principle embedded in:
- Dashboard disclaimer banner
- Flag type labels (anomaly, not fraud)
- Plain-language XAI explanations
- Ethics Policy Section 2
- User acknowledgement form

🔐 Defense in Depth

RBAC enforced at TWO layers:
- Streamlit application layer
- PostgreSQL database engine layer

Aligned with OWASP A01:2025
All access logged to audit_log

⚖️ 7 Bias Sources

Ethics Policy documents:
1. Training data distribution
2. Fiscal year-end temporal
3. Vendor frequency
4. Small vendor
5. False positive concentration
6. Synthetic label evaluation
7. Human confirmation bias

📊 Actionability

False positive handling:
- Thumbs down = not_actionable
- Actionability rate tracked live
- Target: 85% (Herreros-Martinez)
- Declining rate triggers review
- No action before human review

🔒 Data Privacy

GDPR data minimization:
17 of 225 columns selected
Vendor names protected
No external disclosure of flags
Credentials in .env (not GitHub)
90-day rotation in production

📋 NIST AI RMF

Aligned with AI RMF 1.0:
- Fairness: NAICS z-score grouping
- Transparency: XAI explanations
- Reliability: 66 tests verified
- Accountability: full audit_log

Quarterly review schedule planned

5. Course & Program Alignment

IntegriScan integrates all 9 MSIT program courses

MSIT 5210 — Databases

5-table normalized schema
26 indexes, 3 views, RBAC

MSIT 5226 — Foundations of ML

Isolation Forest + z-score
contamination tuning

MSIT 5214 — Algorithms

Z-score computation
chunked ETL complexity

MSIT 5260 — Foundations of HCI

Non-technical dashboard
XAI plain language factors

MSIT 5270 — Cybersecurity

OWASP A01/A02/A09:2025
dual-layer RBAC + audit log

BUS 5115 — Law & Ethics

Ethics Policy v1.0
7 bias sources + NIST RMF

BUS 5611 — Managing Projects

PM Log + Risk Register
8-week milestone tracking

MSIT 5216 — Programming Lang

Python 3.13 + SQL
All 6 code modules

MSIT 5250 — Software Engineering

66-test suite
5-layer architecture + SCM

5. Conclusion & Contributions

What IntegriScan achieved and why it matters

✅ WHAT WAS ACHIEVED

Complete 5-module system built at zero cost:
- 47,540 real contracts processed and analyzed
- 2,397 anomaly flags with dual scores + XAI
- Streamlit dashboard — all 12 peer commitments
- 66/66 tests — 100% pass rate
- All 7 proposal success criteria verified
- Ethics Policy + Data Dictionary + PM Log

🌎 WHY IT MATTERS

Addresses a real governance accountability gap:
- Zero-cost deployment — any government agency
- Reproducible with public open data
- Non-technical auditor interface
- Explainable AI — not a black box
- Human-in-the-loop design throughout
- Post-Capstone commercial roadmap planned

🔬 4 CONTRIBUTIONS TO FIELD

1. Open-source govt procurement anomaly detection using real USASpending.gov data
2. Live actionability feedback as evaluation metric
3. XAI risk factors embedded in auditor dashboard
4. Complete academic + technical documentation package for institutional replication

🧠 5 LESSONS LEARNED

1. Peer review generates concrete improvements
2. Auditor actionability > precision/recall
3. Anomaly ≠ fraud must be in the code itself
4. Configuration management = reproducibility
5. Dataset choice shapes all design decisions

5. Future Work — Post-Capstone Roadmap

IntegriScan Technologies Ltd. — 5-phase development plan

Phase 2
Security Hardening

- MFA — TOTP, SSO, YubiKey
- Server-side session revocation
- Managed secrets store (AWS)
- Second-person config approval

Months 1-3

Phase 3
Infrastructure & Deployment

- Cloud deployment — AWS/Azure
- Docker containerization
- Automated scheduled ETL
- Daily database backup

Months 4-9

Phase 4
ML Enhancement

- Supervised layer from feedback
- Quarterly model review
- Bias analysis by category
- Multi-dataset support

Months 10-15

Phase 5
Product Expansion

- Mobile + WhatsApp alerts
- GamLink super-app (Gambia)
- SaaS for African govts
- ECOWAS market entry

Months 16-24

5 Research Questions identified for academic follow-up | 15 roadmap items documented in IntegriScan_Future_Work_Product_Roadmap_v3.docx

References

APA 7th Edition

Bai, J., & Qiu, T. (2023). *Automatic procurement fraud detection with machine learning*. arXiv. https://arxiv.org/abs/2304.10105

Bennett, L. (2024, August 13). *Software configuration management*. Guru99. https://www.guru99.com/software-configuration-management-tutorial.html

Herreros-Martinez, A., et al. (2025). Applied ML to anomaly detection in enterprise purchase processes. *Information, 16(3)*, 177. https://doi.org/10.3390/info16030177

Letaw, L. (2024). *Handbook of software engineering methods*. Oregon State University. https://open.oregonstate.education/setextbook/

National Institute of Standards and Technology. (2023). *AI Risk Management Framework* (AI RMF 1.0) (NIST AI 100-1). https://doi.org/10.6028/NIST.AI.100-1

OECD. (2025). *Integrity in public procurement*. https://www.oecd.org/en/topics/sub-issues/integrity-in-public-procurement.html

OWASP Foundation. (2026). *OWASP Top 10:2025*. https://owasp.org/Top10/2025/

Summers, B. L. (2020). *Effective methods for software engineering*. Auerbach Publishers.

USASpending.gov. (2026). *FY2023 DoD contracts bulk download*. https://www.usaspending.gov/

Thank you | github.com/sarredrissa-dev/procurement-anomaly-detection | "Follow the money. Flag the risk."